

Helix OTA — Glossary

Field	Value
Revision	1
Created	2026-06-07
Last modified	2026-06-07
Status	active
Status summary	Canonical glossary of domain terms used across the Helix OTA documentation corpus. Each entry gives a precise one- to three-sentence definition plus where the term appears in the corpus. Anchored to the master design (<code>2026-06-07-helix-ota-design.md</code>), the documentation standards, and the research/ stack notes and ADRs.
Issues	Several Android Virtual A/B mechanism details and TUF/Uptane reference-implementation claims originate from version-sensitive AOSP/ upstream prose; where exactness matters they are marked UNVERIFIED here and in the source notes. The exact text/numbering of cited HelixConstitution clauses is UNVERIFIED (per <code>documentation_standards.md</code> §8).
Fixed	N/A (initial revision).
Continuation	Re-confirm Android 15 Virtual A/B figures and RK3588/Orange Pi 5 Max VABc availability against the pinned AOSP/BSP revision; reconcile §11.4.x clause numbers against the authoritative HelixConstitution; add new terms as per-component MVP specs land; cross-link each entry to its primary source section once those specs exist.

Table of contents

The table-of-contents requirement is mandated by HelixConstitution §11.4.61 (UNVERIFIED clause text). Definitions follow the anti-bluff / UNVERIFIED convention of HelixConstitution §7.1 / §11.4.6 / §11.4.123 (UNVERIFIED clause text): no fabricated facts or citations; unconfirmed items are tagged inline with the literal token UNVERIFIED.

1. Scope and conventions
 2. Update mechanism and lifecycle terms
 3. Android device-side terms
 4. Rollout and campaign terms
 5. Server, agent, and artifact terms
 6. Supply-chain trust terms (TUF / Uptane)
 7. Transport, compression, and delivery terms
 8. Platform, storage, and observability terms
 9. Submodule catalogue (canonical names)
 10. Cross-references
-

1. Scope and conventions

This glossary defines every domain term that recurs across the Helix OTA corpus rooted at docs/research/main_specs/. Each entry is: a one- to three-sentence precise definition, followed by **Where it appears** — the corpus location(s) that use the term.

Naming and submodule rules follow documentation_standards.md: only the canonical submodule names from its §9 are used (restated in §9 below); no submodule names are invented. Per the anti-bluff convention, any claim, figure, or citation that has not been confirmed from a real source is marked UNVERIFIED.

2. Update mechanism and lifecycle terms

OTA (Over-the-Air update)

The remote delivery, verification, and installation of a software/firmware update to a device without physical access. Helix OTA is a universal, decoupled OTA system: a Go control plane plus per-OS client SDKs/agents and a dashboard, with Android 15 as the first target OS. **Where it appears:** the project name and brief (../.. / request/helix_ota.md); throughout the master design (2026-06-07-helix-ota-design.md).

A/B (seamless / dual-slot updates)

An update scheme that keeps two complete copies of the bootable partitions (“slots” A and B), writes the update to the currently-inactive slot, then atomically switches to it on reboot; if the new slot fails to boot, the bootloader rolls back to the known-

good slot. **Where it appears:** master design §1, §5, §6 (native Android A/B as the device-side safety core); [../research/stacks/android15-virtual-ab.md](#); [../research/stacks/android-avb-rollback.md](#).

Virtual A/B (VAB) / Virtual A/B Compression (VABc)

Android’s seamless-update mechanism that delivers A/B safety (atomic slot switch, automatic rollback) without physically duplicating every dynamic partition: the update is written to a Copy-on-Write (COW) snapshot, then merged into the base after a confirmed-good boot. VABc adds compression of the COW data (lz4 default, zstd optional, or none), reducing snapshot size (AOSP cites roughly ~45% on a full OTA / ~55% on an incremental OTA — exact figures UNVERIFIED). **Where it appears:** master design §4 diagram (update_engine (Virtual A/B + compression)); [../research/stacks/android15-virtual-ab.md](#) (primary note).

Slot

One of the two complete sets of bootable partitions in an A/B device (slot A or slot B). At any time one slot is *active* (running) and the other is *inactive* (the target the update is written to); a successful update flips which slot is active. **Where it appears:** master design §5 (“applies via update_engine to the inactive slot”); [../research/stacks/android15-virtual-ab.md](#) (atomic slot-switch).

update_engine

The Android Open Source Project (AOSP) daemon that performs A/B and Virtual A/B updates on device: it reads the update payload, writes it to the inactive slot (as COW snapshots under Virtual A/B), coordinates the slot switch, and drives the post-reboot merge. Helix’s design is to orchestrate and wrap update_engine rather than re-implement the apply/merge path. **Where it appears:** master design §4 diagram, §5, §10 (ota-update-engine-bridge wraps it); [../research/stacks/aosp-update-engine.md](#); [../research/stacks/android-update-engine-api.md](#); ADR-0001 ([../research/adr/adr-0001-wrapped-engine.md](#)).

payload.bin

The binary update payload inside an Android OTA package that update_engine consumes; it carries the per-partition operations (and, under Virtual A/B, the COW operations) used to write the inactive slot. (Helix builds the artifact/payload correctly and orchestrates the apply; file-format internals are documented in the AOSP-update-engine notes — UNVERIFIED in exact detail here.) **Where it appears:** referenced in the AOSP update_engine research ([../research/stacks/aosp-update-engine.md](#), [../research/stacks/android-update-engine-api.md](#)); the upload/apply flow in master design §5.

3. Android device-side terms

AVB (Android Verified Boot)

The AOSP mechanism that cryptographically verifies the integrity and authenticity of partitions at boot, establishing a chain of trust from the bootloader. In Helix it is part of the anti-corruption / anti-downgrade posture alongside dm-verity and A/B boot_control. **Where it appears:** master design §4 diagram and §6 (security & trust model); [../research/stacks/android-avb-rollback.md](#).

dm-verity

A Linux device-mapper target that provides transparent, block-level integrity verification of a read-only partition (e.g. /system) against a signed hash tree, so tampered or corrupted blocks are detected on read. Under Virtual A/B the verified partition is mounted through dm-verity layered over dm-user, with I/O served by snapuserd during the snapshot phase. **Where it appears:** master design §4 diagram and §6; [../research/stacks/android15-virtual-ab.md](#) (lifecycle); [../research/stacks/android-avb-rollback.md](#).

boot_control

The Android Hardware Abstraction Layer (HAL) interface used to query and control A/B slot state — which slot is active/bootable, marking a slot successful or unbootable, and selecting the slot to boot next. It is the control surface behind the atomic slot switch and automatic rollback. **Where it appears:** master design §4 diagram and §6; §10 (ota-update-engine-bridge wraps update_engine/boot_control); [../research/stacks/android-avb-rollback.md](#).

update_verifier

The Android component that, on first boot into a newly-updated slot, verifies the integrity of the dm-verity-protected blocks before the boot is marked successful; passing verification is what confirms the update is good (and failing it triggers rollback). **Where it appears:** master design §5 (“reboots → update_verifier confirms → telemetry success/failure with automatic A/B rollback on boot failure”); [../research/stacks/android-avb-rollback.md](#).

rollback-index (anti-rollback / rollback protection)

An AVB-managed monotonic counter that prevents downgrade attacks: the device refuses to boot an image whose rollback index is lower than the value stored in tamper-resistant storage, so an attacker cannot force a return to an older, vulnerable version. (Also discussed in the corpus as bootloader version checks / anti-downgrade.) **Where it appears:** master design §6 (anti-downgrade: “bootloader version checks”); [../research/stacks/android-avb-rollback.md](#).

dm-snapshot / snapuserd / COW (Copy-on-Write)

The device-mapper snapshot machinery behind Virtual A/B. **COW** is the Copy-on-Write area where the update is written instead of overwriting the base; **dm-snapshot** was the older kernel-space snapshot target; **snapuserd** is the userspace daemon (with the dm-user kernel shim) that, from Android 13+, serves snapshot read/write/merge I/O entirely in userspace (Android 15 inherits this model). Merge is resumable across reboots and power-fail safe. **Where it appears:** [../research/stacks/android15-virtual-ab.md](#) §5–§6 (snapshot/COW internals; dm-snapshot deprecation, dm-user, snapuserd).

super / dynamic partitions

Dynamic partitions are resizable logical partitions allocated from a single physical **super** partition, letting partition sizes change via OTA without a fixed physical layout. Virtual A/B leverages this so super need not hold a full resident second slot; transient COW space lives in super if it fits, otherwise spilling to /data. **Where it appears:** [../research/stacks/android15-virtual-ab.md](#) §3 (“Dynamic Partitions & the super Partition”) and §8 (super sizing).

4. Rollout and campaign terms

Staged / phased rollout

A deployment strategy that releases an update to progressively larger fractions of the fleet (e.g. 5% → 10% → 30% → ... → 100%) instead of all at once, so problems surface on a small cohort first. In Helix the staged rollout engine lands in the 1.0.1-staged-rollout phase; the 1.0.0-MVP deploys all-at-once. **Where it appears:** brief ([../request/helix_ota.md](#)); master design §5, §8 (staged rollout engine), §11 (1.0.1-staged-rollout/).

Cohort / canary

A **cohort** is the specific subset of devices targeted by a given rollout phase. A **canary** is an initial, small cohort that receives the update first to detect failures before wider exposure. (In the corpus, cohort selection is the engine’s per-phase device set; “canary” denotes the small early phase — explicit per-phase canary configuration is design-level and detailed in the 1.0.1 rollout specs, UNVERIFIED beyond the staged-rollout note.) **Where it appears:** master design §8 (“targets the phase cohort”, “deterministic cohort selection”); [../research/additions_synthesis.md](#).

Halt-on-failure

The rollout-engine safety rule that automatically halts or pauses a campaign when telemetry from the current phase breaches a configured error threshold, preventing a bad update from propagating to more devices. The engine advances only when a

success threshold is met within the phase duration. **Where it appears:** master design §4 diagram (“Rollout engine (staged %, halt-on-failure)”) and §8 (“halts/ pauses on error-threshold breach”); §9 (telemetry drives the halt logic).

5. Server, agent, and artifact terms

Control plane

The server side of Helix OTA: a Go + Gin service that handles auth, artifact intake/validation, release/deploy, the rollout engine, the device registry, and telemetry ingest — the system that operators interact with via the dashboard. It is deliberately decoupled from the OS-specific apply mechanism. **Where it appears:** master design §1 (vision), §4 (Control Plane subgraph); brief ([../request/helix_ota.md](#)).

Device agent

The on-device client (Android: a Kotlin Multiplatform / KMP agent) that polls the control plane, downloads updates, re-verifies them, applies them via the OS mechanism (`update_engine`), and reports status/telemetry back. Helix’s MVP poll default is 15 minutes plus jitter (configurable). **Where it appears:** master design §4 diagram (OTA agent (KMP)), §5; §10 (`ota-android-agent` new submodule).

Artifact

A versioned, validated update blob managed by the system (for Android, the signed OTA .zip plus its mandatory hash file). Artifacts are stored in object storage (MinIO/S3), validated on upload (structure, hash, signature, version monotonicity, target compatibility), and referenced by releases and deployments. **Where it appears:** brief ([../request/helix_ota.md](#)); master design §4 (“Artifact intake + validation”), §5, §7 (artifacts table), §10 (`ota-artifact-validator`).

Manifest

The structured metadata describing an update/release — schema, version, target compatibility, status/event enums — carried as part of the shared wire contract between server and agents (the `ota-protocol` submodule defines the manifest schema). In the TUF/Uptane sense, “manifest” also denotes the signed metadata that lists the authorized images (see §6). **Where it appears:** master design §10 (`ota-protocol`: “Shared wire types, manifest schema, status/event enums”); supply-chain trust ADR-0002 ([../research/adr/adr-0002-supply-chain-trust.md](#)).

6. Supply-chain trust terms (TUF / Uptane)

TUF (The Update Framework)

A framework for securing software-update systems through signed, role-separated metadata that remains resilient even if parts of the update infrastructure are compromised. In Helix, TUF/Uptane is forward-path hardening for 1.0.1+; signing interfaces are designed so it can drop in without rework. **Where it appears:** master design §6 (“forward path”), §16 (ADR-0002); [../research/stacks/tuf-go-tuf.md](#); ADR-0002 ([../research/adr/adr-0002-supply-chain-trust.md](#)).

TUF roles: root / targets / snapshot / timestamp

The four core TUF metadata roles, each with its own signing key(s): **root** is the trust anchor that delegates and distributes the keys for the other roles; **targets** signs the actual update files (and delegations); **snapshot** signs a consistent set of the current metadata versions to prevent mix-and-match attacks; **timestamp** is a frequently-resigned, short-lived role that lets clients detect that they are seeing current metadata (freshness / replay protection). **Where it appears:** [../research/stacks/tuf-go-tuf.md](#); [../research/stacks/uptane.md](#) §5 (Roles, Metadata & Verification); ADR-0002.

Uptane

A security framework built on top of TUF for automotive/embedded fleets, specifying how update metadata is signed, distributed, and verified to resist a compromised update infrastructure. Helix’s recommendation is to adopt Uptane’s metadata/security model selectively, not as the primary update transport/mechanism. **Where it appears:** [../research/stacks/uptane.md](#) (primary note); ADR-0002.

Uptane Director repository / Image repository

Uptane’s two-repository architecture. The **Director repository** decides, per device, which specific images that device should install and produces signed targets metadata tailored to it (it knows the fleet/inventory). The **Image repository** holds and signs the full catalogue of available images and their metadata (it does not know which device gets what). A client verifies both, gaining defense-in-depth. **Where it appears:** [../research/stacks/uptane.md](#) §4 (“Architecture: Director + Image Repositories”); ADR-0002. Note: no production Android/AOSP update_engine + Uptane integration is confirmed to ship today (UNVERIFIED), per the Uptane note’s Issues.

7. Transport, compression, and delivery terms

Delta update

An update that ships only the difference between the device's current version and the target version (an incremental payload) rather than the full image, reducing download size at the cost of build/apply complexity. In Helix, delta updates are out of scope for 1.0.0-MVP and are the subject of a dedicated ADR. **Where it appears:** master design §1 (non-goals), §16 (ADR-0005); ADR-0005 ([../research/adr/adr-0005-delta-updates.md](#)); incremental-OTA references in [../research/stacks/android15-virtual-ab.md](#).

HTTP/3 / QUIC

HTTP/3 is the HTTP version that runs over **QUIC**, a UDP-based transport with built-in encryption and reduced head-of-line blocking versus TCP. Helix mandates HTTP/3 (QUIC) as the primary transport via the `http3` submodule, with automatic fallback to HTTP/2. **Where it appears:** master design §2 (D6), §3 (transport), §4 (Control Plane), §6; §16 (ADR-0004); ADR-0004 ([../research/adr/adr-0004-transport.md](#)).

Brotli

A general-purpose lossless compression algorithm used for HTTP content compression; Helix mandates Brotli for content compression with negotiated fallback to gzip for older clients. **Where it appears:** master design §2 (D6), §3 (transport), §4 (Control Plane header); ADR-0004 ([../research/adr/adr-0004-transport.md](#)).

8. Platform, storage, and observability terms

MinIO

An S3-compatible object storage server; in Helix it is the artifact blob store (MinIO/S3) for OTA packages, accessed via the `Storage` submodule. **Where it appears:** master design §2 (D7), §3 (persistence), §4 (Data plane), §5.

OpenTelemetry

A vendor-neutral standard and toolset for collecting telemetry (traces, metrics, logs). Helix emits telemetry via the `observability` submodule and surfaces it through Prometheus/Grafana; this telemetry drives both fleet-health reporting and the rollout halt-on-failure logic. **Where it appears:** master design §2 (D7), §3 (observability), §4 (Data plane), §9 (telemetry & observability).

9. Submodule catalogue (canonical names)

These are the only submodule names referenced in this glossary; they are the verified canonical set from [documentation_standards.md](#) §9 and are not invented or renamed.

Core / backend submodules: auth, security, database, Storage, observability, eventbus, ratelimiter, middleware, http3, mdns, recovery, Herald, config, discovery, cache, docs_chain / Document / Formatters, containers.

KMP (Kotlin Multiplatform) submodules: Auth-KMP, Security-KMP, Storage-KMP, Config-KMP.

Glossary terms that map to these submodules:

Term / capability	Canonical submodule
Object storage (MinIO/S3 artifacts)	Storage
OpenTelemetry / observability	observability
HTTP/3 (QUIC) transport	http3
Alerting / health surfacing	Herald
Multi-format export pipeline	docs_chain / Document / Formatters
Containerized infrastructure substrate	containers
Optional caching layer	cache
Android device agent building blocks	Auth-KMP, Security-KMP, Storage-KMP, Config-KMP

The new OTA-specific submodules named in the master design (ota-protocol, ota-artifact-validator, ota-rollout-engine, ota-update-engine-bridge, ota-android-agent, ota-telemetry-schema) are proposed repositories defined in master design §10, not part of the verified existing catalogue above; their final list is confirmed in the MVP spec before creation.

10. Cross-references

- Documentation rules, metadata/ToC mandate, and the canonical submodule list: [documentation_standards.md](#).
- System vision, decisions, architecture, and where most terms originate: [2026-06-07-helix-ota-design.md](#).
- Operator brief (source of the OTA / rollout / safety requirements): [../../request/helix_ota.md](#).
- Android device-side mechanism terms: [../research/stacks/android15-virtual-ab.md](#), [../research/stacks/android-avb-rollback.md](#), [../research/stacks/aosp-update-engine.md](#), [../research/stacks/android-update-engine-api.md](#).
- Supply-chain trust terms: [../research/stacks/tuf-go-tuf.md](#), [../research/stacks/uptane.md](#), [../research/adr/adr-0002-supply-chain-trust.md](#).

- Transport / delta terms: ../research/adr/adr-0004-transport.md, ../research/adr/adr-0005-delta-updates.md.